

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ing. en Tecnologías De La Información Y La Innovación Digital

REPORTE DE PRÁCTICA

Estados en Jetpack Compose: manejo de recomposición con State, remember y rememberSaveable

Asignatura: Desarrollo de aplicaciones móviles

Alumno: Lorenzana Hernández Aarón Dahir

Matrícula: 2531126341

Grupo: TIID 4-02

Profesor(a): MIGUEL ANGEL MONTOYA CERRO

Pachuca de Soto, Hidalgo, México, 29/09/2026

Índice de contenido

1. Introducción 1

1.1 Contextualización 1

1.2 Planteamiento del problema u objeto de estudio 1

1.3 Objetivos 2

1.4 Justificación 2

1.5 Estructura del documento 3

2. Marco teórico 4

2.1 Definición de conceptos clave 4

2.2 Teorías y modelos relevantes 5

2.3 Antecedentes 6

2.4 Fuentes de información 6

3. Desarrollo 7

3.1 Creación de función Composable con variable estándar 7

3.2 Implementación de mutableStateOf y remember 9

3.3 Optimización sintáctica con el delegado by 11

3.4 Pruebas de pérdida de estado por cambios de configuración 13

3.5 Persistencia del estado utilizando rememberSaveable 15

3.6 Análisis de resultados 17

4. Conclusiones 18

4.1 Aportes 18

4.2 Limitaciones 18

4.3 Recomendaciones y proyecciones 18

5. Referencias 19

6. Anexos 20

Anexo A. Datos del video consultado 20

Índice de ilustraciones

Ilustración 1: Árbol del proyecto en Android Studio con el archivo MainActivity.kt 7

Ilustración 2: Declaración de una variable normal (var counter = 0) en Compose 8

Ilustración 3: Vista inicial del emulador con el contador en 0 8

Ilustración 4: Falla de actualización visual en el emulador tras pulsar el botón 9

Ilustración 5: Implementación de remember { mutableStateOf(0) } en el código 10

Ilustración 6: Ajuste del código para acceder a la propiedad .value del estado 10

Ilustración 7: Actualización exitosa en el emulador tras recomponer (5 clics) 11

Ilustración 8: Sustitución de la asignación por el delegado de propiedad by 11

Ilustración 9: Importación manual de las funciones getValue y setValue 12

Ilustración 10: Código optimizado incrementando la variable directamente sin .value 12

Ilustración 11: Rotación del dispositivo (estado perdido, contador regresa a 0) 13

Ilustración 12: Implementación de rememberSaveable para sobrevivir a la rotación 14

Ilustración 13: Importación de la librería rememberSaveable en el encabezado 15

Ilustración 14: Acumulación de clics en la vista vertical del emulador 16

Ilustración 15: Conservación del estado al rotar el emulador a formato horizontal 16

Índice de tablas

Tabla 1: Comparación de manejo de UI entre sistema imperativo y declarativo 5

Tabla 2: Herramientas y tecnologías utilizadas en la práctica 7

Tabla 3: Funciones clave para el manejo de estado en Jetpack Compose 15

Tabla 4: Relación entre objetivos específicos y evidencias del desarrollo 17

1. Introducción

1.1 Contextualización

El desarrollo de interfaces de usuario modernas en Android ha evolucionado desde el paradigma imperativo tradicional, basado en archivos XML, hacia un enfoque declarativo impulsado por Jetpack Compose. En un sistema declarativo, la interfaz de usuario se considera inmutable una vez generada; la única forma de actualizarla es llamando nuevamente a la función que la dibuja, pasándole nuevos datos. Este proceso de redibujado se conoce como recomposición.

Para que Jetpack Compose sepa cuándo debe desencadenar una recomposición, requiere que las variables de datos estén envueltas en objetos observables. Comprender cómo almacenar y preservar estos datos en la memoria temporal de las funciones composables es el núcleo de la interactividad en este moderno framework de Android.

1.2 Planteamiento del problema u objeto de estudio

El objeto de estudio de esta práctica es la correcta gestión del estado, basado en el tutorial "Estados (STATES) en JETPACK COMPOSE" del canal Programación by AristiDevs (Guimerá Orozco, 2021). El problema principal radica en que, a diferencia de Java o Kotlin clásico, actualizar una variable local entera (ej. `var counter = 0`) dentro de una función `@Composable` no provoca que la pantalla se actualice, debido a que el framework no puede "observar" dicho cambio.

Además, aunque se convierta la variable en un estado observable con `mutableStateOf`, las recomposiciones constantes reinicializan la variable a cero a menos que se utilice la función `remember`. Finalmente, se aborda el problema de la pérdida de datos cuando el usuario rota el dispositivo, destruyendo la `Activity`, lo cual se soluciona migrando a `rememberSaveable`.

1.3 Objetivos

Objetivo general. Implementar interactividad en una pantalla de Android mediante la manipulación de estados en Jetpack Compose, solucionando los problemas de recomposición, pérdida de valor temporal y destrucción por rotación del dispositivo.

Objetivos específicos.

1. Comprobar empíricamente que una variable estándar no actualiza la interfaz gráfica al cambiar su valor en un entorno declarativo.
2. Integrar `mutableStateOf` para convertir una variable de tipo entero en un estado observable por el motor de Compose.
3. Aplicar la función `remember` para persistir el valor de la variable a través de las múltiples recomposiciones (redibujados) de la pantalla.
4. Optimizar la sintaxis del código utilizando la delegación de propiedades (`by`) para evitar el uso redundante del atributo `.value`.
5. Evidenciar la pérdida del estado de la interfaz al simular un cambio de configuración (rotación del emulador).
6. Solucionar la pérdida de estado provocada por la destrucción de la Activity empleando `rememberSaveable`.

1.4 Justificación

Académicamente, esta práctica es vital dentro de la asignatura de Desarrollo de aplicaciones móviles, pues representa el puente entre el diseño estático de vistas y la programación lógica de aplicaciones dinámicas e interactivas. Sin el dominio del estado, es imposible capturar inputs del usuario, contadores, o información proveniente de una base de datos.

A nivel profesional, dominar `remember` y `rememberSaveable` previene bugs críticos relacionados con la rotación de pantalla, el cambio a modo oscuro o la multitarea, asegurando una experiencia de usuario robusta y libre de inconsistencias, un estándar exigido en la industria del software.

1.5 Estructura del documento

El documento se divide en seis secciones. Tras esta introducción, la segunda sección expone el marco teórico sobre la inmutabilidad y la recomposición. La tercera sección detalla el desarrollo práctico paso a paso, ilustrando la evolución del código desde una variable inútil hasta un estado persistente. Finalmente, se presentan las conclusiones, las referencias consultadas y los anexos con la información técnica del videotutorial.

2. Marco teórico

En este apartado se definen los conceptos intrínsecos al motor de redibujado de Jetpack Compose, fundamentales para comprender la naturaleza de las interfaces declarativas.

2.1 Definición de conceptos clave

Recomposición. Es el proceso mediante el cual Jetpack Compose vuelve a invocar a las funciones `@Composable` con datos actualizados para emitir una nueva estructura de la interfaz. De manera inteligente, Compose solo redibuja las funciones cuyos parámetros o estados internos hayan cambiado.

State (Estado). Es cualquier valor observable que puede cambiar con el tiempo y que dicta lo que se muestra en la interfaz (por ejemplo, el texto en una caja de búsqueda o el número de un contador). Compose proporciona la interfaz `State<T>` y su contraparte modificable `MutableState<T>`.

remember. Es un bloque de memoria temporal en una función Composable. Evalúa la expresión que contiene durante la composición inicial, almacena el resultado y lo devuelve durante las recomposiciones posteriores, evitando que las variables se restablezcan a su valor predeterminado.

rememberSaveable. Similar a `remember`, pero guarda el valor en un `Bundle` del sistema operativo. Esto permite que el estado sobreviva a la muerte y recreación de la `Activity` que ocurre durante los cambios de configuración, como la rotación

de pantalla o el redimensionamiento de ventanas.

2.2 Teorías y modelos relevantes

Paradigma Imperativo vs Declarativo. En el sistema XML clásico (imperativo), los componentes de la interfaz como `TextView` conservan su propio estado interno (por ejemplo, su propiedad de texto). El desarrollador debe buscar el componente usando `findViewById` y mutarlo manualmente con `setText()`.

En el modelo declarativo de Compose, los componentes son funciones sin estado que simplemente dibujan lo que se les pasa como parámetro. Por tanto, el estado debe ser elevado y gestionado por la aplicación. La Tabla 1 contrasta estas metodologías.

Tabla 1: Comparación de manejo de UI entre sistema imperativo y declarativo

Característica	Sistema XML (Imperativo)	Jetpack Compose (Declarativo)
Almacenamiento de estado	Propiedad interna del widget.	Externo a la función, inyectado como parámetro.
Método de actualización	Mutación manual (ej. `textView.text = "Hola"`).	Mutación del estado observable; Compose redibuja solo.
Flujo de datos	Bidireccional y propenso a inconsistencias.	Unidireccional (UDF - Unidirectional Data Flow).
Supervivencia a rotación	Requiere código complejo de `onSaveInstanceState`.	Se resuelve de forma concisa con `rememberSaveable`.

2.3 Antecedentes

Previo a Jetpack Compose, el estado se gestionaba de forma rudimentaria interceptando los ciclos de vida de la actividad o delegándolo por completo a la capa de arquitectura mediante `ViewModel` y `LiveData`. Aunque el uso de

`ViewModel` sigue siendo la mejor práctica para la lógica de negocio, Jetpack Compose introdujo `State` y `remember` para manejar lógicas exclusivas de la interfaz sin ensuciar la arquitectura con variables triviales de UI.

2.4 Fuentes de información

El marco teórico se fundamenta en la guía oficial de Google sobre el estado y Jetpack Compose, así como en los conceptos analizados por Guimerá Orozco (2021) en su curso práctico para AristiDevs.

3. Desarrollo

A continuación se documenta el progreso metodológico de la práctica, evidenciando las iteraciones del código fuente para el manejo de estados. La Tabla 2 lista el entorno de trabajo.

Tabla 2: Herramientas y tecnologías utilizadas en la práctica

Herramienta	**Versión / Detalle**	**Uso en la práctica**
Android Studio	IDE Oficial Android	Edición de archivos Kotlin y emulación (Ilustración 1).
Jetpack Compose	UI Toolkit	Provisión de `Button`, `Text`, `Column` y `State`.
Kotlin Delegation	Delegado `by`	Simplificación de lectura y escritura del estado observable.
Emulador (AVD)	Dispositivo Virtual	Prueba de la interacción táctil y rotación del sensor de orientación.

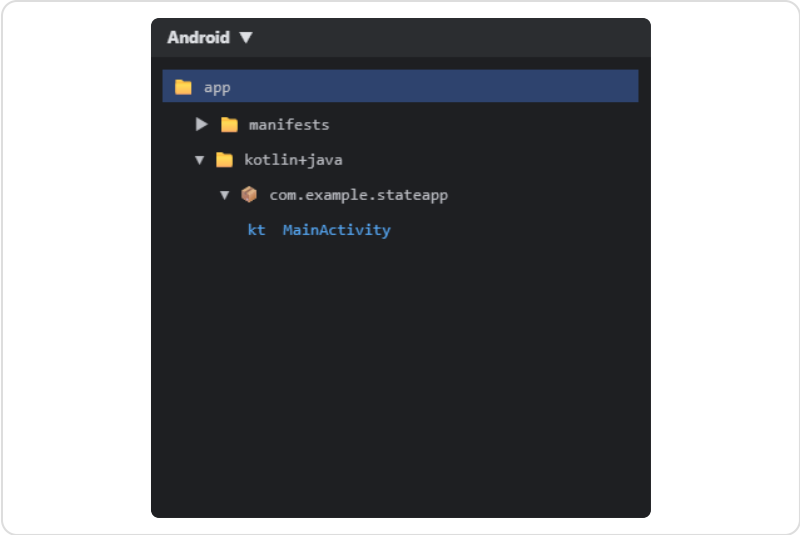


Ilustración 1: Árbol del proyecto en Android Studio con el archivo MainActivity.kt

3.1 Creación de función Composable con variable estándar

Para iniciar, se creó una función `@Composable` llamada `MyStateExample()`. Dentro de ella, se declaró una variable entera convencional `var counter = 0` y se estructuró una columna centrada con un botón que incrementaba esta variable al ser pulsado, y un texto que mostraba su valor (Ilustración 2 y Bloque de código 1).

Bloque de código 1. Declaración de un contador con variable normal (incorrecto)

```
@Composable
fun MyStateExample() {
    var counter = 0

    Column(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Button(onClick = { counter += 1 }) {
            Text(text = "Pulsar")
        }
        Text(text = "He sido pulsado $counter veces")
    }
}
```

> **Descripción:** El botón ejecuta el incremento de la variable en memoria, pero el motor de Compose no se entera de este cambio.



Ilustración 2: Declaración de una variable normal (var counter = 0) en Compose

Al ejecutar la aplicación, la pantalla inicializó correctamente mostrando "He sido pulsado 0 veces" (Ilustración 3). No obstante, al pulsar reiteradamente el botón, el texto en pantalla nunca se actualizó, evidenciando que Compose necesita ser notificado explícitamente para recomponer la vista (Ilustración 4).



Ilustración 3: Vista inicial (contador 0)



Ilustración 4: Falla de redibujo al pulsar

3.2 Implementación de mutableStateOf y remember

Atendiendo a las instrucciones del tutorial, la solución inicial fue envolver el valor entero dentro de `mutableStateOf(0)`, lo que creó un canal de suscripción para Compose. Sin embargo, esto no bastaba, ya que cada redibujado reinicializaría el objeto. Por lo tanto, se anidó dentro de la función `remember { }` para guardar su memoria (Ilustración 5 y Bloque de código 2).

Bloque de código 2. Uso combinado de remember y mutableStateOf

```
1  @Composable
2  fun MyStateExample() {
3      var counter = remember { mutableStateOf(0) }
4
5      Column(modifier = Modifier.fillMaxSize() /*...*/) {
6          Button(onClick = { counter.value += 1 }) {
7              Text(text = "Pulsar")
8          }
9          Text(text = "He sido pulsado ${counter.value} veces")
10     }
11 }
```

> **Descripción:** En las líneas 6 y 9 es obligatorio acceder a la propiedad `.value` porque la variable es ahora de tipo `MutableState<Int>` y no un entero primitivo (Ilustración 6).



Ilustración 5: Implementación remember



Ilustración 6: Ajuste a .value

Al compilar nuevamente y realizar clics sobre el botón, se observó cómo la interfaz comenzó a reaccionar correctamente a los cambios de estado, actualizando el recuento visualmente en pantalla gracias a la recomposición (Ilustración 7).



Ilustración 7: Actualización exitosa en el emulador tras recomponer (5 clics)

3.3 Optimización sintáctica con el delegado by

Para evitar la tediosa y repetitiva llamada a `.value` en el código, el video tutorial enseña a utilizar el delegado de propiedades de Kotlin (palabra clave `by`). Esto desempaqueta el valor observable para tratarlo directamente como un entero (Ilustraciones 8 y 10, y Bloque de código 3).

Para que esta sintaxis sea admitida por el IDE Android Studio, fue fundamental añadir manualmente los *imports* de `getValue` y `setValue` en la cabecera del archivo, los cuales no se auto-importaban en las versiones de Kotlin utilizadas en el tutorial de 2021 (Ilustración 9).

Bloque de código 3. Refactorización usando delegación de propiedades (by)

```
1  import androidx.compose.runtime.getValue
2  import androidx.compose.runtime.setValue
3
4  @Composable
5  fun MyStateExample() {
6      var counter by remember { mutableStateOf(0) }
7
8      Column(/*...*/) {
9          Button(onClick = { counter += 1 }) {
10             Text(text = "Pulsar")
11         }
12         Text(text = "He sido pulsado $counter veces")
13     }
14 }
```

> **Descripción:** El uso de by en la línea 6 elimina la necesidad de escribir `.value`.



Ilust. 8: Delegado by



Ilust. 9: Imports manuales



Ilust. 10: Limpieza código

3.4 Pruebas de pérdida de estado por cambios de configuración

Al tener la aplicación funcional, se sometió al escenario más temido de Android: la rotación de pantalla. Tras presionar el botón 5 veces y voltear el emulador al formato horizontal, se evidenció que el contador regresó a 0 (Ilustración 11).

Esto ocurre porque `remember` guarda su caché en la memoria local del entorno Composable de la Actividad actual. Al rotar el teléfono, el sistema operativo destruye la Actividad (Activity Lifecycle) para recrearla con las dimensiones apaisadas, evaporando así cualquier memoria volátil de `remember`.



Ilustración 11: Rotación del dispositivo (estado perdido, contador regresa a 0)

Tabla 3: Funciones clave para el manejo de estado en Jetpack Compose

Función	Comportamiento	Sobrevive Recomposición	Sobrevive Rotación
Variable (`var x = 0`)	Se reinicializa siempre.	❌ No	❌ No
`remember { }`	Cachea a nivel local.	✅ Sí	❌ No
`rememberSaveable { }`	Serializa en el Bundle OS.	✅ Sí	✅ Sí

3.5 Persistencia del estado utilizando rememberSaveable

La solución definitiva empleada en el videotutorial fue sustituir la función `remember` por `rememberSaveable` (Bloque de código 4, Ilustraciones 12 y 13).

Bloque de código 4. Estado persistente a cambios de configuración

```
import androidx.compose.runtime.saveable.rememberSaveable
//... resto de imports ...

@Composable
fun MyStateExample() {
    var counter by rememberSaveable { mutableStateOf(0) }

    Column(modifier = Modifier.fillMaxSize(), /* ... */ ) {
        Button(onClick = { counter += 1 }) {
            Text(text = "Pulsar")
        }
        Text(text = "He sido pulsado $counter veces")
    }
}
```

> **Descripción:** Al usar `rememberSaveable`, el entero es empaquetado por Android y recuperado cuando la Activity termina de rotar y redibujarse.



Ilustración 12: Cambio a rememberSaveable

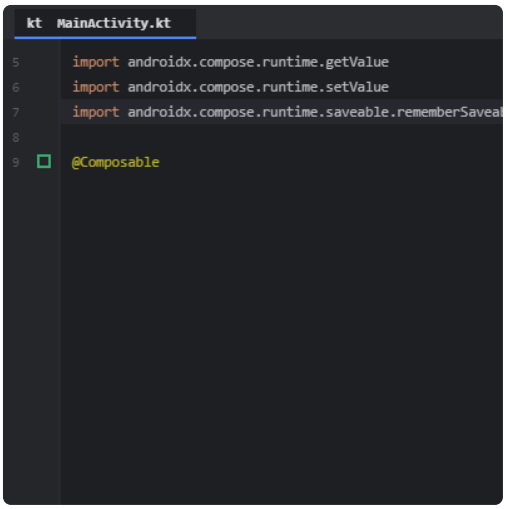


Ilustración 13: Import en encabezado

Al repetir las pruebas en el emulador, se realizaron nuevamente 5 clics en orientación vertical (Ilustración 14). Posteriormente, se activó la rotación del teléfono a formato horizontal (*Landscape*); esta vez, la interfaz redibujó el botón pero conservó el conteo en 5, confirmando la supervivencia del estado (Ilustración 15).



Ilustración 14: Vista vertical (5 clics)



Ilustración 15: Conservación al rotar

3.6 Análisis de resultados

El ejercicio práctico evidenció las diversas capas de gestión de variables dentro del motor declarativo de Android. Se validó que las interfaces en Compose son pasivas hasta que un estado observable ordene su re-renderizado. La refactorización utilizando delegados optimizó el flujo de lectura del código, y la adopción final de `rememberSaveable` resolvió con elegancia las debilidades del ciclo de vida clásico de Android, logrando componentes reactivos.

4. Conclusiones

4.1 Aportes

Se logró comprender y asimilar la arquitectura unidireccional de flujo de datos (UDF) impuesta por Jetpack Compose. El aporte más significativo fue interiorizar la delegación con by y distinguir en qué casos es suficiente `remember` y cuándo es

mandatario `rememberSaveable`.

4.2 Limitaciones

Esta práctica se limitó a estados locales primitivos (tipo `Int`) administrados dentro de una misma función. En aplicaciones complejas, el estado no debe permanecer encapsulado en la capa de vista, sino que debe "elevarse" (State Hoisting).

4.3 Recomendaciones y proyecciones

Como paso subsiguiente para escalar estos conocimientos, se recomienda investigar el patrón de diseño "State Hoisting" (elevación de estado), el cual consiste en extraer el `remember` hacia componentes padres para que múltiples elementos hijos, y finalmente ViewModels, puedan leer y alterar el estado compartido de manera limpia y testeable.

5. Referencias

Android Developers. (s. f.-a). *Aspectos básicos del diseño de Compose*. Google. https://developer.android.com/develop/ui/compose/layouts/basics?hl=es-419

Android Developers. (s. f.-b). *El estado y Jetpack Compose*. Google. https://developer.android.com/develop/ui/compose/state?hl=es-419

Guimerá Orozco, A. [Programación by AristiDevs]. (2021, 21 de octubre). *Estados (STATES) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]* [Video]. YouTube. https://www.youtube.com/watch?v=CwR1VUeRjvs

6. Anexos

Anexo A. Datos del video consultado

Título: Estados (STATES) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]

Canal: Programación by AristiDevs (Mobile Dev by AristiDevs)

Enlace al video: https://www.youtube.com/watch?v=CwR1VUeRjvs

Fecha de publicación original: 21 de octubre de 2021

Temas técnicos abordados: Recomposición, creación de variables de estado (`MutableState`), persistencia en memoria local con `remember`, delegación mediante `by`, y la solución a los cambios de configuración con `rememberSaveable`.